

# SSH Banner with Ansible

## Ad-Hoc Commands Lab

Configure, validate, reload, verify, and roll back safely

### TARGET

`three_tier_app`

### CONTROL NODE

`ansible-server`

### MANAGED NODES

`node1 node2 node3`

English study deck

# Learning path

A safe sequence for an SSH configuration change

**01** Understand the target and modules

**02** Create the banner file

**03** Manage sshd\_config with regex

**04** Validate, reload, and test

**05** Confirm idempotency and roll back

# Lab environment

Your Rocky Linux control node and three managed nodes

## CONTROL NODE

**ansible-server**

192.168.1.233

**ansibleadmin**

```
/home/ansibleadmin/  
automation
```

## MANAGED NODES

**node1**

192.168.1.154

Rocky Linux

**node2**

192.168.1.185

Rocky Linux

**node3**

192.168.1.190

Rocky Linux

# Safety rules before changing SSH

Protect access while you test configuration changes

**1** Keep the current SSH session open

**2** Use a second terminal for testing

**3** Start with `--limit node1`

**4** Create a backup before editing

**5** Validate before reload

**6** Prefer reload over restart

**Tip:** `--limit node1`

# Modules used in this lab

Each module owns a specific part of the workflow

**ping**

**Test Ansible connectivity**

**copy**

Create the banner file

**command**

Run safe commands without a shell

**stat**

Inspect file metadata

**lineinfile**

Manage the Banner directive

**shell**

Use pipes for effective-config checks

**service**

Reload sshd

**file**

Remove the banner file during cleanup

# Project configuration and connectivity

Run from /home/ansibleadmin/automation

```
[defaults]
inventory = ./inventory/nodes
remote_user = ansibleadmin
private_key_file = /home/ansibleadmin/.ssh/ansible-
key
host_key_checking = False
```

```
[privilege_escalation]
become_method = sudo
become_user = root
become_ask_pass = False
```

```
cd /home/ansibleadmin/automation
ansible three_tier_app -m ping
ansible three_tier_app --list-hosts
```

# Create the SSH banner

The template variable gives each host a unique message

```
ansible three_tier_app -b -m copy -a \
'content="*****\nWelcome to {{ inventory_hostname }} for Ansible
practice\n*****\n" dest=/etc/ssh/banner.txt owner=root group=root mode=0644'
```

**RESULT ON EACH NODE**

Welcome to node1 for Ansible practice

# Inspect the banner file

Use command for content and stat for metadata

## CONTENT

```
ansible three_tier_app -b -m command -a \  
"cat /etc/ssh/banner.txt"
```

**changed: false**

## METADATA

```
ansible three_tier_app -b -m stat -a \  
"path=/etc/ssh/banner.txt"
```

**owner=root mode=0644**

# Manage sshd\_config

lineinfile replaces a commented or active Banner directive

```
ansible three_tier_app -b --limit node1 -m lineinfile -a \
'path=/etc/ssh/sshd_config \
regexp="^\s*#\s*Banner\s+.*$" \
line="Banner /etc/ssh/banner.txt" state=present backup=yes \
validate="/usr/sbin/sshd -t -f %s"'
```

**DESIRED STATE**

**Banner /etc/ssh/banner.txt**

After node1 succeeds, repeat without --limit.

# Regex anatomy

Read the pattern from left to right

```
^\s*#?\s*Banner\s+.*$
```

^

Start of line

\s\*

Zero or more spaces

#?

Optional comment character

\s\*

Optional spaces again

Banner

Exact directive name

\s+

One or more spaces

.\*

Any remaining value

\$

End of line

# What the regex matches

— Anchors prevent partial directive names from matching

## MATCHES

```
#Banner none
# Banner /etc/issue.net
Banner none
  Banner /etc/ssh/banner.txt
```

`^ ... $`

## DOES NOT MATCH

```
BannerFile /tmp/banner.txt
MyBanner none
AllowBanner yes
```

**Directive name is not exact**

# Multiple matching lines

lineinfile normally changes the last match

**If sshd\_config contains several Banner lines, lineinfile changes the last matching line. Use the replace module when every match must be changed or removed.**

## Validate the complete SSH configuration

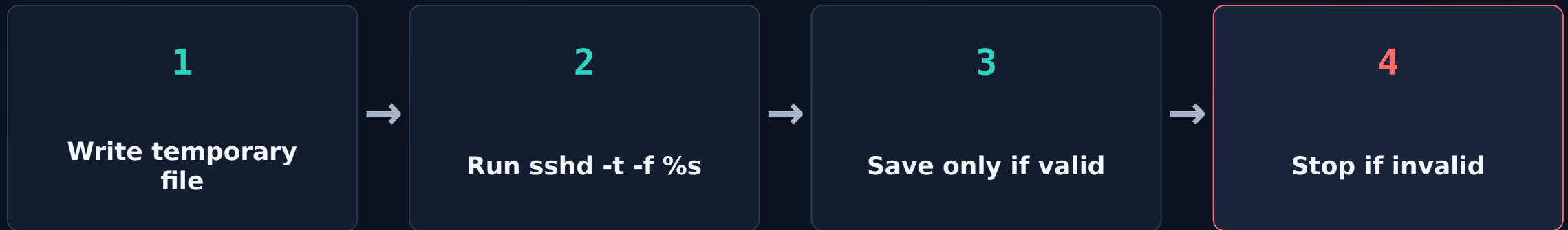
```
ansible three_tier_app -b -m command -a "/usr/sbin/sshd -t"
```

```
ansible three_tier_app -b -m shell -a \  
"/usr/sbin/sshd -T | grep '^banner '"
```

# Validate before applying

Ansible tests a temporary configuration file first

SAFE FLOW



```
validate="/usr/sbin/sshd -t -f %s"
```

# Reload and verify sshd

Apply the valid configuration without dropping existing sessions

```
ansible three_tier_app -b -m service -a \  
"name=sshd state=reloaded"
```

```
ansible three_tier_app -m command -a \  
"systemctl is-active sshd"
```

```
ansible three_tier_app -m shell -a \  
"pgrep -af sshd"
```

# Test from an SSH client

The banner appears before authentication

```
ssh -i ~/.ssh/ansible-key ansibleadmin@node1  
ssh -i ~/.ssh/ansible-key ansibleadmin@node2  
ssh -i ~/.ssh/ansible-key ansibleadmin@node3
```

## EXPECTED EXPERIENCE

**Banner → authentication prompt → shell after successful login**

# Idempotency

The second run should not make another change

FIRST RUN

**changed: true**

SECOND RUN

**changed: false**

Run the copy and lineinfile commands again. A stable configuration should report no change.

# Rollback and cleanup

Return SSH to the original state and remove lab artifacts

```
ansible three_tier_app -b -m lineinfile -a \
'path=/etc/ssh/sshd_config regexp="^\s*#?\s*Banner\s+.*$" \
line="#Banner none" state=present backup=yes \
validate="/usr/sbin/sshd -t -f %s"'
```

```
ansible three_tier_app -b -m service -a "name=sshd state=reloaded"
ansible three_tier_app -b -m file -a "path=/etc/ssh/banner.txt state=absent"
```

**Test SSH again after rollback.**

# Review checklist

A complete ad-hoc configuration workflow



**Connectivity confirmed**



Banner file created



Directive managed with regex



Configuration validated



sshd reloaded and checked



SSH login tested



Second run is idempotent



Rollback completed